```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.models import Sequential
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0
# reshape for CNN
X_train_cnn = X_train.reshape(-1,28,28,1)
X_test_cnn = X_test.reshape(-1,28,28,1)

model = Sequential([

    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),

    Flatten(),

    Dense(128, activation='relu'),
    Dense(10, activation='softmax')

])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train_cnn, y_train,
          epochs=10,
          batch_size=64,
          validation_split=0.1)

test_loss, test_acc = model.evaluate(X_test_cnn, y_test)

print("Test accuracy:", test_acc)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

```
 Total params: 225,034 (879.04 KB)
 Trainable params: 225,034 (879.04 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 44s 50ms/step - accuracy: 0.9502 - loss: 0.1687 - val_accuracy: 0.9837 - val_loss: 0
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 49ms/step - accuracy: 0.9841 - loss: 0.0513 - val_accuracy: 0.9873 - val_loss: 0
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 42s 50ms/step - accuracy: 0.9896 - loss: 0.0342 - val_accuracy: 0.9897 - val_loss: 0
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 80s 48ms/step - accuracy: 0.9921 - loss: 0.0244 - val_accuracy: 0.9895 - val_loss: 0
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 48ms/step - accuracy: 0.9938 - loss: 0.0188 - val_accuracy: 0.9895 - val_loss: 0
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 48ms/step - accuracy: 0.9954 - loss: 0.0139 - val_accuracy: 0.9905 - val_loss: 0
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 48ms/step - accuracy: 0.9964 - loss: 0.0117 - val_accuracy: 0.9895 - val_loss: 0
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 48ms/step - accuracy: 0.9972 - loss: 0.0087 - val_accuracy: 0.9902 - val_loss: 0
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 48ms/step - accuracy: 0.9966 - loss: 0.0090 - val_accuracy: 0.9915 - val_loss: 0
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 48ms/step - accuracy: 0.9977 - loss: 0.0070 - val_accuracy: 0.9897 - val_loss: 0
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.9916 - loss: 0.0327
Test accuracy: 0.991599977016449
```

```
#DENSE LAYER(128) REMOVED
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.models import Sequential
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0
# reshape for CNN
X_train_cnn = X_train.reshape(-1,28,28,1)
X_test_cnn = X_test.reshape(-1,28,28,1)

model = Sequential([

    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),

    Flatten(),

    Dense(10, activation='softmax')

])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

```
model.summary()

model.fit(X_train_cnn, y_train,
          epochs=10,
          batch_size=64,
          validation_split=0.1)

test_loss, test_acc = model.evaluate(X_test_cnn, y_test)

print("Test accuracy:", test_acc)
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_3 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten_1 (Flatten)	(None, 1600)	0
dense_2 (Dense)	(None, 10)	16,010

Total params: 34,826 (136.04 KB)
Trainable params: 34,826 (136.04 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 46ms/step - accuracy: 0.9355 - loss: 0.2163 - val_accuracy: 0.9823 - val_loss: 0
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 46ms/step - accuracy: 0.9801 - loss: 0.0671 - val_accuracy: 0.9845 - val_loss: 0
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 39s 46ms/step - accuracy: 0.9848 - loss: 0.0507 - val_accuracy: 0.9887 - val_loss: 0
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 46ms/step - accuracy: 0.9883 - loss: 0.0389 - val_accuracy: 0.9895 - val_loss: 0
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 38s 45ms/step - accuracy: 0.9905 - loss: 0.0310 - val_accuracy: 0.9845 - val_loss: 0
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 38s 45ms/step - accuracy: 0.9911 - loss: 0.0280 - val_accuracy: 0.9895 - val_loss: 0
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 46ms/step - accuracy: 0.9929 - loss: 0.0219 - val_accuracy: 0.9865 - val_loss: 0
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 38s 45ms/step - accuracy: 0.9940 - loss: 0.0182 - val_accuracy: 0.9892 - val_loss: 0
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 39s 46ms/step - accuracy: 0.9950 - loss: 0.0154 - val_accuracy: 0.9897 - val_loss: 0
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 38s 45ms/step - accuracy: 0.9957 - loss: 0.0134 - val_accuracy: 0.9908 - val_loss: 0
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - accuracy: 0.9878 - loss: 0.0396
Test accuracy: 0.9878000020980835
```

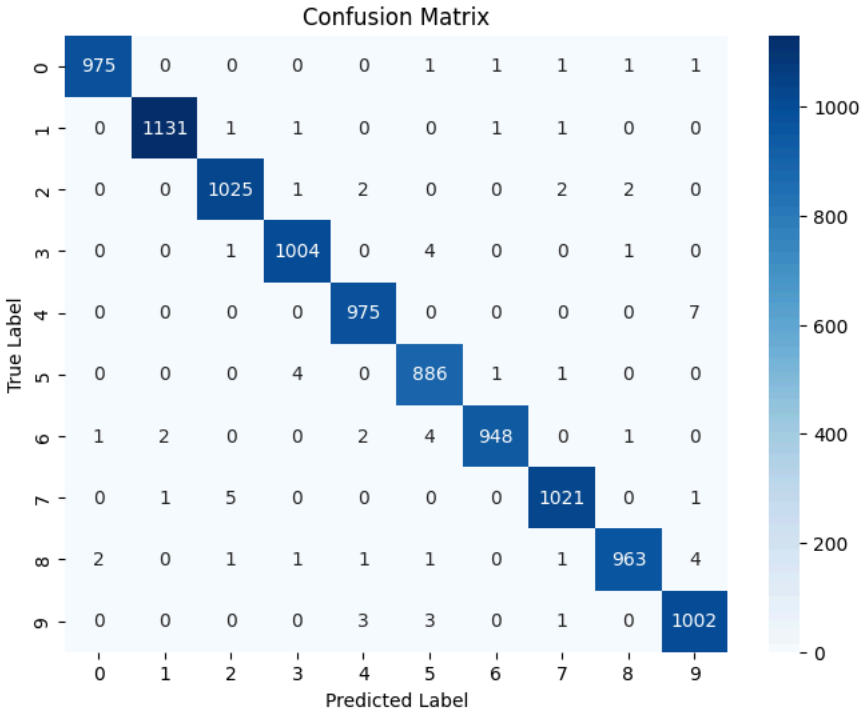
```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.models import Sequential
import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0
# reshape for CNN
X_train_cnn = X_train.reshape(-1,28,28,1)
X_test_cnn = X_test.reshape(-1,28,28,1)

model = Sequential([

    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),

    Flatten(),

    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
```

```
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train_cnn, y_train,
          epochs=10,
          batch_size=64,
          validation_split=0.1)

test_loss, test_acc = model.evaluate(X_test_cnn, y_test)

print("Test accuracy:", test_acc)
import matplotlib.pyplot as plt
history = model.fit(X_train_cnn, y_train,
                    epochs=10,
                    batch_size=64,
                    validation_split=0.1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train Accuracy', 'Validation Accuracy'])

plt.show()
```

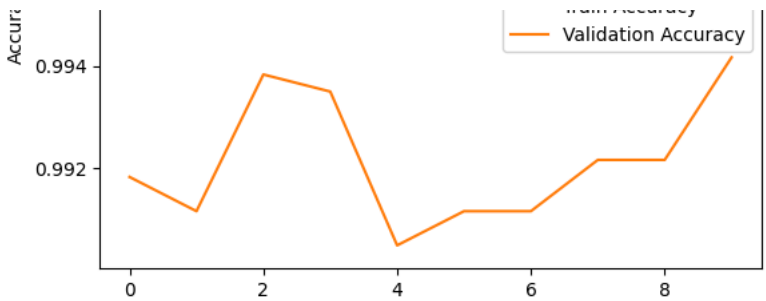
```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pa
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

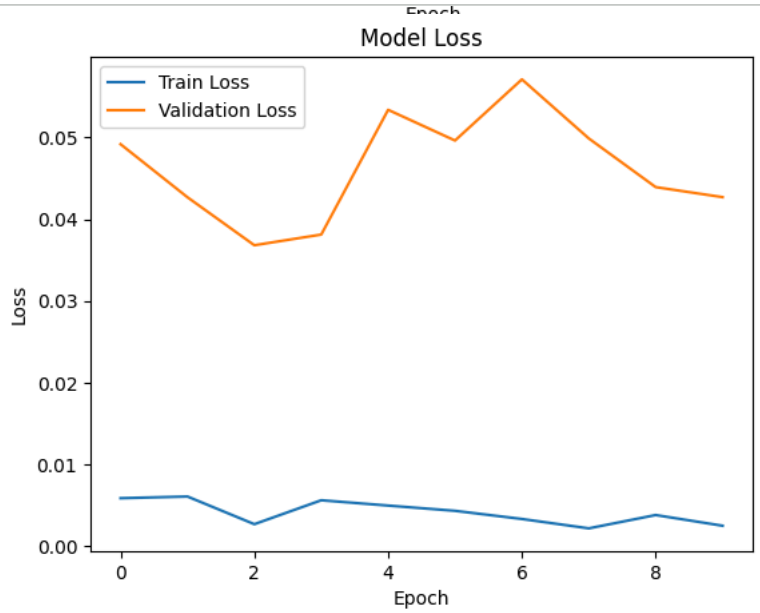
```
# ONCFUSION MATRIX
from sklearn.metrics import confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Predict classes
y_pred = model.predict(X_test_cnn)
y_pred_classes = np.argmax(y_pred, axis=1)

# Create confusion matrix
cm = confusion_matrix(y_test, y_pred_classes)

# Plot confusion matrix
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Confusion Matrix")

plt.show()
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9924 - loss: 0.0243 - val_accuracy: 0.9887 - val_loss: 0
al_accuracy: 0.9907 - val_loss: 0
al_accuracy: 0.9898 - val_loss: 0
al_accuracy: 0.9893 - val_loss: 0
al_accuracy: 0.9907 - val_loss: 0
al_accuracy: 0.9913 - val_loss: 0
al_accuracy: 0.9892 - val_loss: 0
al_accuracy: 0.9918 - val_loss: 0
al_accuracy: 0.9912 - val_loss: 0
al_accuracy: 0.9938 - val_loss: 0
al_accuracy: 0.9935 - val_loss: 0
al_accuracy: 0.9905 - val_loss: 0
al_accuracy: 0.9912 - val_loss: 0
al_accuracy: 0.9912 - val_loss: 0
al_accuracy: 0.9922 - val_loss: 0
al_accuracy: 0.9922 - val_loss: 0
Epoch 10/10
```




```
import matplotlib.pyplot as plt

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])

plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train Loss', 'Validation Loss'])

plt.show()
```






```
# removing seconf convolution operation from cnn
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.models import Sequential
import tensorflow as tf
from tensorflow.keras.datasets import mnist
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# Reshape for CNN
X_train_cnn = X_train.reshape(-1,28,28,1)
X_test_cnn = X_test.reshape(-1,28,28,1)

# CNN Model (Single Convolution Layer)
model = Sequential([

    Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
    MaxPooling2D((2,2)),

    Flatten(),

    Dense(128, activation='relu'),
    Dense(10, activation='softmax')

])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

model.fit(X_train_cnn, y_train,
          epochs=10,
          batch_size=64,
          validation_split=0.1)

test_loss, test_acc = model.evaluate(X_test_cnn, y_test)

print("Test accuracy:", test_acc)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
flatten (Flatten)	(None, 5408)	0
dense (Dense)	(None, 128)	692,352
dense_1 (Dense)	(None, 10)	1,290

```
 Total params: 693,962 (2.65 MB)
 Trainable params: 693,962 (2.65 MB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 8s 6ms/step - accuracy: 0.9463 - loss: 0.1850 - val_accuracy: 0.9793 - val_loss: 0.0
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9814 - loss: 0.0613 - val_accuracy: 0.9798 - val_loss: 0.0
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9872 - loss: 0.0409 - val_accuracy: 0.9883 - val_loss: 0.0
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9911 - loss: 0.0277 - val_accuracy: 0.9878 - val_loss: 0.0
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9942 - loss: 0.0194 - val_accuracy: 0.9873 - val_loss: 0.0
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9956 - loss: 0.0140 - val_accuracy: 0.9870 - val_loss: 0.0
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9966 - loss: 0.0110 - val_accuracy: 0.9882 - val_loss: 0.0
```

```
# removing dense layer of 128 neurons from cnn
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
from tensorflow.keras.models import Sequential
import tensorflow as tf
from tensorflow.keras.datasets import mnist
import numpy as np

# Load dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```